Kubernetes Complete Detailed Notes (Faculty Style + Beginner Friendly + Interview Ready)

What is Kubernetes?

Kubernetes is a powerful open-source platform for automating the deployment, scaling, and operation of application containers.

Kubernetes is also known as:

K8S

Kubernetes can manage containerized applications.

Real-World Problem Before Kubernetes

Suppose we have:

- One application
- Running on one server/node

If heavy traffic comes on application:

- That single node/server may fail
- Application may go down

So immediately we need:

- Backup nodes
- Multiple nodes
- Load distribution

Question:

How to control and manage multiple nodes?

Answer:

Kubernetes

Kubernetes Creates Cluster

When we deploy Kubernetes:

- We get a Cluster

Cluster contains:

1. Master Node (Control Plane)
2. Worker Nodes

What is Cluster?

Cluster means:

- Group of systems/nodes working together.

Kubernetes creates, deploys and manages clusters.

Kubernetes is a Container Orchestration Tool

Kubernetes is a Container Orchestration tool.

What is Orchestration?

Orchestration means:

- Managing containers across multiple servers/nodes automatically.

Example:

- Scheduling containers
- Scaling containers
- Managing containers
- Monitoring containers

Important Kubernetes Features

1. Orchestration

Clustering any number of containers on different hardware/nodes.

---

2. Autoscaling

Automatically increases or decreases application resources according to traffic/load.

---

3. Auto Healing

If containers crash:

- Kubernetes automatically creates new containers.

Same concept exists in Docker Swarm also.

---

4. Load Balancing

Distributes traffic among multiple Pods/containers.

---

5. Roll Back

Rollback means:

- Going to previous stable versions if new version fails.

Docker vs Kubernetes

Docker Swarm and Kubernetes are both container orchestration platforms, but they have different strengths and use cases.

Docker Swarm

Docker Swarm is:

- Docker’s native clustering and orchestration tool
- Designed to manage and scale containerized applications

Docker Swarm is:

- Easier to use
- Better for smaller applications

Kubernetes

Kubernetes is:

- More robust
- Better for large-scale applications
- More powerful for enterprise environments

Important Understanding

Docker is used to create containers
Kubernetes is used to manage them

Real-Life Explanation Given by Faculty

Docker carries containers as shown in logo.

Kubernetes acts like steering/control system for Docker containers.

Limitations of Docker Swarm Compared to Kubernetes

1. Scalability Limitation

Docker Swarm is generally considered less scalable than Kubernetes.

While Swarm can handle clusters with reasonable number of nodes and services:

- Kubernetes is designed for very large-scale deployments
- Kubernetes can manage thousands of nodes and Pods

---

2. Feature Limitation

Swarm provides:

- Basic load balancing
- Basic routing

But lacks advanced capabilities provided by Kubernetes.

---

3. Security Limitation

Kubernetes offers:

- Role-Based Access Control (RBAC)
- Network Policies
- Security Policies

Docker Swarm security features are less comprehensive.

---

4. Community Limitation

Docker Swarm has:

- Smaller community
- Less industry adoption

Compared to Kubernetes.

Kubernetes Cluster Architecture

Kubernetes architecture is designed to manage containerized applications across cluster of machines efficiently.

It follows:

Master-Worker Architecture

Where:

- Master Nodes control and manage Worker Nodes.

Kubernetes Architecture Components

1. Master Nodes (Control Plane)
2. Worker Nodes
3. Pods
4. Networking
5. Volumes
6. Namespaces
7. Services

Memory Shortcut

MW - PNVNS

M = Master Nodes
W = Worker Nodes
P = Pods
N = Networking
V = Volumes
N = Namespaces
S = Services

Master Nodes (Control Plane)

The Master Node is responsible for managing the Kubernetes cluster.

It coordinates:

- Scheduling
- Scaling
- Maintaining desired state
- Cluster management activities

Key Components of Master Node

1. API Server (kube-apiserver)

Definition

API Server is the entry point for all REST commands used to control the cluster.

It communicates with user:

- Takes commands
- Executes commands
- Gives output

Real-Life Analogy

API Server = Receptionist

Important Statement

kubectl is the command used to communicate with API Server

---

kubectl

kubectl is the command-line tool for Kubernetes.

If we want to execute commands:

- We use kubectl.

Example

kubectl get pods

---

2. Scheduler (kube-scheduler)

Definition

Scheduler is responsible for placing Pods onto Nodes within cluster based on:

- Resource availability
- Constraints
- CPU
- Memory

This kube-scheduler communicates with Nodes.

Faculty Line

Scheduler will take action

Meaning:

- Scheduler decides where Pod should run.

Real-Life Analogy

Scheduler = Hotel room allocator.

---

3. Controller Manager (kube-controller-manager)

Definition

This component runs various controllers that regulate the state of cluster.

Examples:

- Node Controller
- Replication Controller

Responsibilities

Node Controller

Handles node failures.

Replication Controller

Maintains correct number of Pods.

Important Faculty Point

Controller Manager controls Kubernetes objects:

- Network
- Services
- Nodes

---

4. etcd

Definition

etcd is a key-value store used by Kubernetes to store all cluster data.

Stores:

- Configuration data
- State information
- Metadata

Faculty Definition

etcd = Database of the Cluster

Critical Point

etcd is:

Single source of truth for cluster state

---

5. Cloud Controller Manager (Optional)

Definition

This component interacts with underlying cloud infrastructure.

Handles:

- Load Balancers
- Volumes
- Cloud resources

Works in:

- Public cloud
- Private cloud

Important Point

Responsible to ensure:

Actual state = Desired state

Worker Nodes

Worker Nodes are systems where:

- Applications run
- Pods run
- Containers run

Components of Worker Node

1. kubelet
2. kube-proxy
3. container-runtime

4. POD

---

1. kubelet

Definition

kubelet is an agent in Worker Node.

Faculty Explanation

kubelet informs all activities to Master

Responsibilities

- Monitors Pods
- Ensures containers are running properly
- Communicates with Master Node

Important Point

kubelet makes sure:

Containers are running inside Pods

---

2. kube-proxy

Definition

kube-proxy deals with:

- Networking
- IPs
- Networks
- Ports

Responsibilities

Maintains:

- Network rules
- Pod communication
- Service communication

Faculty Line

kube-proxy maintains network rules for communication with Pods

---

3. container-runtime

Definition

Container runtime is tool responsible for running containers.

Examples:

- Docker
- containerd
- CRI-O

POD

What is POD?

Pod is:

The smallest and simplest Kubernetes object

Official Style Definition

A Pod represents:

- A single instance of running process in cluster.

It can run:

- One container
- Multiple containers

And containers inside Pod:

- Share same resources

Important Faculty Explanation

Within POD, we have container

Very Important Point

Pods are:

Ephemeral

Meaning:

- Can be created anytime
- Can be destroyed anytime

Pods are unit of scaling in Kubernetes.

Most Important Kubernetes Concept

Kubernetes does not understand Containers
Kubernetes understands only PODS

Important Statements

Smallest object Kubernetes can create is POD

POD contains Docker containers

Cluster is bunch of PODS

Faculty Style Explanation

By using Kubernetes:

- We form cluster
- Group of PODS with Docker containers

K8S schedules, runs and manages isolated PODS.

Key Networking Concepts

1. Cluster IP

Definition

Cluster IP is:

- Internal IP address assigned to service within cluster.

Used for:

- Internal communication only.

---

2. NodePort

Definition

NodePort exposes service on:

- Static port
- Node IP

Allows external access.

---

3. LoadBalancer

Definition

LoadBalancer provisions a load balancer in supported cloud environments to expose services externally.

Mostly used in:

- AWS
- Azure
- Google Cloud

Volumes

Definition

Kubernetes Volumes provide persistent storage to containers within Pods.

Allows:

- Data persistence across Pod restarts.

Storage Backends

Volumes can use:

- Local storage
- Cloud storage
- AWS EBS

Important Point

Even if Pod restarts:

- Data remains safe.

Namespaces

Definition

Namespaces are way to divide cluster resources between multiple users.

Purpose of Namespaces

They provide:

- Scope for names

Meaning:

- Multiple resources can have same name
- In different namespaces

Example

dev namespace
test namespace
production namespace

Services

Definition

Services are used to access applications running inside Pods.

Applications can be accessed using:

- Cluster IP (internal access)
- Load Balancer (external access)

Why Services Needed?

Pods are temporary.

Services provide:

- Stable access point
- Stable communication

Kubernetes Cluster Creation Types

In Kubernetes we need to create Cluster.

Cluster can be created in 2 ways:

---

1. Self Managed

We need to create and manage cluster ourselves.

Examples

Minikube

Single node cluster.

Used for:

- Practice
- Learning
- Local testing

---

kubeadm

Multi-node cluster (manual setup).

---

kops

Multi-node cluster (automation).

---

2. Cloud-Based Kubernetes

Cloud providers manage Kubernetes clusters.

Cloud Kubernetes Services

AWS

EKS = Elastic Kubernetes Service

Azure

AKS = Azure Kubernetes Service

Google

GKS = Google Kubernetes Service

Brief History of Kubernetes

Google's Borg

Origins of Kubernetes started from Google internal system called:

Borg

Borg was:

- Large-scale cluster management system
- Developed internally by Google

Used to:

- Deploy applications
- Manage infrastructure
- Scale applications across thousands of servers

Efficiently.

Omega

After Borg:
Google developed:

Omega

Omega introduced:

- Flexible architecture
- Better scheduling
- Better resource management
- Better orchestration concepts

Birth of Kubernetes

Kubernetes officially launched in:

2014

Initially developed by Google engineers:

- Joe Beda
- Brendan Burns
- Craig McLuckie

Kubernetes Design

Kubernetes was designed using lessons learned from:

- Borg
- Omega

But Kubernetes was intended to be:

- Open-source

- Publicly available

Open Source and Community

From beginning:
Kubernetes was designed as open-source project.

Released under:

Apache 2.0 License

Allowing:

- Anyone to use
- Anyone to modify
- Community contribution

CNCF Partnership

In 2015:
Kubernetes was donated to:

CNCF
(Cloud Native Computing Foundation)

This helped Kubernetes gain:

- Huge adoption
- Enterprise trust
- Strong open-source ecosystem

Final Quick Revision

Question	Answer
What is Kubernetes?	Container orchestration platform

Smallest object in Kubernetes?	POD
Kubernetes understands what?	PODS
etcd role?	Database of cluster
kubectl role?	Communicate with API Server
kubelet role?	Agent in worker node
kube-proxy role?	Networking
Scheduler role?	Places Pods on nodes
Cluster contains what?	Master + Worker Nodes

Ultimate Beginner-Friendly Summary

Docker creates containers
Kubernetes manages containers
Pods contain containers
Cluster contains Pods
Master controls Workers
Services expose applications
Volumes store data
Namespaces divide resources

Kubernetes Manifest File Notes (Faculty Style + Beginner Friendly + Interview Ready)

What is a Manifest File in Kubernetes?

Manifest file means:

- YAML file used to define Kubernetes objects.

Using manifest files we can create:

- Pods
- Deployments
- Services
- ReplicaSets

- Secrets
- Jobs
- Ingress

Mandatory Parameters in Manifest File

In every Kubernetes manifest file, we mostly use these mandatory parameters:

apiVersion:
kind:
metadata:
spec:

---

1. apiVersion

What is apiVersion?

apiVersion specifies:

- The API version used for Kubernetes object.

Different Kubernetes objects use different API groups/versions.

Faculty Explanation

Depending on Kubernetes object we want to create,
there is a corresponding code library we want to use.

apiVersion refers to Code Library.

Example

apiVersion: apps/v1

This means:

- We are using apps API group version v1.

Common apiVersion Examples

Kubernetes Object	apiVersion
POD	v1
Service	v1
Namespace	v1
Secrets	v1
ReplicaSet	apps/v1
Deployment	apps/v1
Jobs	batch/v1

Command to Check API Versions

kubectl api-versions

This command shows:

- All API versions supported by cluster.

---

2. kind

What is kind?

kind refers to:

Which Kubernetes object we want to create

Examples

kind: Pod
kind: Deployment
kind: Service
kind: Ingress
kind: Job

Explanation

If we write:

kind: Pod

Kubernetes understands:

- We want to create Pod.

If we write:

kind: Deployment

Kubernetes understands:

- We want to create Deployment object.

---

3. metadata

What is metadata?

metadata contains:

- Additional information about Kubernetes object.

Examples of metadata

- Name

- Labels
- Namespace
- Annotations

Example

```
metadata:
  name: nginx-pod
  labels:
    app: nginx
```

Explanation

Here:

- nginx-pod = Object name
- app: nginx = Label

---

4. spec

What is spec?

spec contains:

- Actual configuration of object.

It defines:

- What object should do
- Container details
- Image
- Ports
- Volumes
- Resources

Example

```
spec:
  containers:
  - name: nginx
    image: nginx
```

POD Notes

POD Definition

Pods are:

The smallest and simplest Kubernetes objects

Important Definition

A Pod represents:

- Single instance of running process inside cluster.

Important Characteristics of POD

1. Pods are Ephemeral

Pods are:

Short living objects

Meaning:

- Pods can be created anytime
- Pods can be destroyed anytime

---

2. Pod Can Have Single or Multiple Containers

Mostly:

- One Pod contains one container.

But if required:

- We can create multiple containers inside same Pod.

Important Faculty Point

Containers inside same POD can share:
- Same network namespace
- Same storage volumes

---

3. Pod Configuration

While creating Pod:

- We must specify image
- Configuration
- Resource limits

---

4. Very Important Kubernetes Concept

Kubernetes cannot communicate with Containers.
Kubernetes communicates only with PODS.

Ways to Create POD

We can create Pod in two ways:

1. Imperative (Command Method)

Using direct commands.

---

2. Declarative (Manifest File Method)

Using YAML files.

Important Faculty Point

Declarative approach can be reused.
In real-time industry mostly Declarative method is used.

Imperative Method

What is Imperative Method?

In Imperative method:

- We directly run kubectl commands.

Example

Create Pod

kubectl run pod1 --image nginx

Explanation

Creates:

- Pod with name pod1
- Using nginx image

Get Pods

kubectl get pods

OR

kubectl get po

Get Detailed Information

kubectl get pod -o wide

This gives:

- Pod IP
- Node name
- Additional details

Describe Pod

kubectl describe pod pod1

Shows:

- Events
- Configuration
- Container details
- Errors

Delete Pod

kubectl delete pod pod1

Declarative Method

What is Declarative Method?

In Declarative method:

- We write YAML manifest file.

Then apply using:

kubectl apply -f filename.yaml

Why Declarative Method Preferred?

Because:

- Reusable
- Easy to manage
- Easy to modify
- Used in real projects
- Infrastructure as Code approach

Example POD YAML File

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod

spec:
  containers:
  - name: nginx-container
    image: nginx
```

Explanation of Above YAML

Field	Meaning
apiVersion: v1	Using v1 API
kind: Pod	Creating Pod
metadata	Additional details
name	Name of Pod

spec	Actual configuration
containers	Container details
image: nginx	Pull nginx image

Important Interview Questions

Question:

Why does Kubernetes use POD instead of directly managing containers?

Answer:

Kubernetes manages Pods because:

- Pod is smallest deployable unit
- Pod can contain one or multiple containers
- Containers inside Pod can share storage and networking

Question:

What is difference between Imperative and Declarative?

Answer:

Imperative	Declarative
Command based	YAML based
Fast for testing	Used in real projects
Hard to reuse	Reusable
Manual approach	Infrastructure as Code

Quick Revision

apiVersion = API library/version
kind = Kubernetes object type
metadata = Additional information
spec = Actual configuration

Final Beginner-Friendly Summary

Manifest file is YAML file used to create Kubernetes objects.

apiVersion tells Kubernetes which API version to use.

kind tells Kubernetes what object to create.

metadata stores additional information.

spec contains actual configuration.

Pods are smallest Kubernetes objects.

Kubernetes communicates with Pods, not containers.

Pods can be created using:
1. Imperative method
2. Declarative method

Vi pod.yml

```
apiVersion: v1
kind: Pod

metadata:
  name: pod1

spec:
  containers:
    - name: cont1
      image: nginx
```